Comprendre le TP cardiaque, partie par partie

Explications simples des 7 parties du code et de leur utilité

> **Le fil conducteur.** On apprend à une machine à reconnaître un cœur « dilaté » sur une image d'échographie. Même démarche que pour tout projet sérieux : regarder les images (**1**), mettre un examen de côté (**2**), transformer les images en chiffres utilisables et comparer des modèles (**3**), régler le meilleur (**4**), passer l'examen une seule fois (**5**), décider à partir de quelle probabilité on alerte (**6**) et vérifier que le modèle regarde au bon endroit (**7**).

Trois idées à connaître avant de commencer

- **Une image, c'est un tableau de nombres.** Notre image 64 × 64 contient 4 096 nombres entre 0 (noir) et 1 (blanc). Le modèle ne « voit » que ces nombres.
- **Les images sont simulées.** Le fichier `echo_gen.py` dessine une cavité sombre (le sang) entourée d'un muscle clair, puis ajoute le bruit granuleux des ultrasons. C'est un exercice : on peut faire la méthode sans données de patients.
- **Ici la classe positive est « pathologique » (1).** Contrairement au TP précédent, une erreur grave (rater un cœur malade) est donc un **FN**.

Partie 1 · Exploration des images

> **L'idée en une phrase.** regarder les images avant de modéliser.

Explication simple

On génère 600 images, puis on vérifie leur taille, leurs valeurs et la répartition des classes. On affiche surtout quelques exemples de chaque classe et leur **image moyenne** : en superposant toutes les images d'une classe, on voit la forme « typique ». Ici, la cavité du cœur pathologique est plus grande et la paroi plus fine.

Le code, ligne par ligne

Ligne de code	Ce qu'elle fait, en mots simples
`X, y = make_dataset(n=600, prevalence=0.3, seed=0)`	Crée 600 images et leurs étiquettes. `prevalence=0.3` : environ 30 % de cas pathologiques, `seed=0` : même résultat à chaque exécution.
`print("Images :", X.shape, ...)`	Affiche (600, 64, 64) : 600 images de 64 × 64 pixels, et l'intervalle des valeurs (0 à 1).
`{0: (y==0).sum(), 1: (y==1).sum()}`	Compte les normales (440) et les pathologiques (160).
`ax[0, i].imshow(X[y == 0][i], cmap="gray")`	Affiche la i-ème image normale en niveaux de gris. `X[y == 0]` garde seulement les images de classe 0.
`X[y == 0].mean(0)`	Calcule l'image moyenne de la classe (moyenne pixel par pixel sur toutes les images).

> **Ce qu'on obtient.** (600, 64, 64) · valeurs entre 0 et 1 · 440 normales / 160 pathologiques.

> **À quoi ça sert dans le TP.** Comprendre ce qu'on cherche à distinguer et choisir les bonnes mesures. Les classes sont inégales (27 % de pathologiques) : on suivra la **sensibilité**, le **F1** et l'**AUC** plutôt que l'accuracy seule.

Piège à éviter. Passer directement au modèle sans regarder les images. Sur de vraies échographies, ce regard permet de repérer des images floues, des annotations incrustées ou des différences d'appareil qui fausseraient tout.

Partie 2 · Découpage train / test

L'idée en une phrase. garder des images « jamais vues » pour l'évaluation finale.

Explication simple

Comme pour un examen blanc sous clé : 80 % des images (480) servent à apprendre, 20 % (120) restent cachées jusqu'à la partie 5. Le découpage est **stratifié** : la proportion de cas pathologiques reste la même des deux côtés.

Le code, ligne par ligne

Ligne de code	Ce qu'elle fait, en mots simples
`train_test_split(X, y, test_size=0.2, ...)`	Sépare 20 % des images pour le test.
`stratify=y`	Garde environ 27 % de pathologiques dans le train comme dans le test.
`random_state=42`	Fixe le hasard pour retrouver le même découpage à chaque fois.
`round(100*y_tr.mean(), 1)`	Les étiquettes valent 0 ou 1 : leur moyenne est la proportion de pathologiques (26,7 %).

Ce qu'on obtient. Train 480 images · Test 120 images.

À quoi ça sert dans le TP. Tout ce qui suit (comparaison, réglage, seuil) n'utilise que le train. Le test ne sert qu'à la fin.

Piège à éviter. Sur de vraies données, plusieurs images peuvent venir du même patient. Si elles se retrouvent à la fois dans train et test, la note est gonflée : on découpe alors **par patient**, jamais par image.

Partie 3 · Caractéristiques et comparaison de modèles

L'idée en une phrase. transformer chaque image en liste de chiffres utiles, puis comparer trois modèles.

Explication simple

Donner 4 096 pixels bruts à un modèle simple est peu efficace : un petit décalage change tous les nombres. On utilise donc **HOG** (*histogramme de gradients orientés*) : on découpe l'image en cases de 16 × 16 pixels, et dans chaque case on note la **direction des contours**. Le cœur dilaté a des contours plus grands et plus espacés, ce que HOG capture. Ensuite on compare trois modèles par validation croisée : régression logistique, SVM (qui cherche la meilleure frontière entre les classes) et forêt aléatoire (beaucoup d'arbres qui votent).

Le code, ligne par ligne

Ligne de code	Ce qu'elle fait, en mots simples
`def extraire_hog(images): ...`	Applique HOG à chaque image et empile les résultats : on passe de (n, 64, 64) à (n, 324).

Ligne de code	Ce qu'elle fait, en mots simples
`pixels_per_cell=(16, 16), cells_per_block=(2, 2)`	Taille des cases (16 × 16) et regroupement par blocs de 2 × 2 cases pour normaliser la lumière.
`FunctionTransformer(extraire_hog)`	Transforme notre fonction en étape de pipeline.
`Pipeline([("hog",...),("sc",...),("m", ...)])`	Enchaîne : HOG → mise à l'échelle → modèle. On peut alimenter le pipeline directement avec des images.
`CalibratedClassifierCV(SVC(), ensemble=False)`	Le SVC seul ne donne pas de vraies probabilités. Cette enveloppe les calibre, ce qui servira pour le seuil (partie 6).
`cross_val_score(m, X_tr, y_tr, cv=cv, scoring="f1")`	5 tours d'apprentissage/validation sur le train, avec le score F1.

Ce qu'on obtient. Logistique 0,889 · SVM 0,908 · Forêt 0,867 (F1 en validation croisée).

À quoi ça sert dans le TP. Choisir le modèle sur des faits, sans toucher au test. Le SVM sort en tête (F1 ≈ 0,908), suivi de la logistique (0,889) et de la forêt (0,867).

Piège à éviter. Faire la mise à l'échelle avant le pipeline, sur toutes les images : de l'information du pli de validation fuiterait dans l'entraînement.

Partie 4 · Optimisation du SVM

L'idée en une phrase. régler les deux boutons du SVM, C et gamma.

Explication simple

C règle la tolérance aux erreurs : petit = frontière simple, grand = frontière qui veut tout classer juste. **gamma** règle la « portée » de chaque image dans la décision : petit = influence large, grand = influence très locale. On teste 4 × 3 = 12 combinaisons, chacune avec 5 plis : 60 entraînements.

Le code, ligne par ligne

Ligne de code	Ce qu'elle fait, en mots simples
`{"m__estimator__C": [...], "m__estimator__gamma": [...]}`	La grille des valeurs à essayer. Le nom suit l'emboîtement : étape « m » → calibrateur → SVC (« estimator »).
`GridSearchCV(models["SVM"], grille, cv=cv, scoring="f1")`	Essaie toutes les combinaisons avec validation croisée.
`grid.fit(X_tr, y_tr)`	Lance la recherche sur le train uniquement.
`grid.best_params_ / best_score_`	Meilleure combinaison (C = 1, gamma = 0,005) et son F1 (0,91).

Ce qu'on obtient. C = 1, gamma = 0,005, F1 de validation croisée = 0,91.

À quoi ça sert dans le TP. Récupérer un modèle prêt à servir : `grid.best_estimator_`. Ici le gain est minime (0,908 → 0,91) : le réglage n'est pas ce qui limite la performance, c'est le bruit des images.

Piège à éviter. Croire qu'un réglage fin va tout résoudre. Quand le score ne bouge pas, mieux vaut améliorer les données (plus d'images, meilleures caractéristiques).

Partie 5 · Évaluation finale

L'idée en une phrase. ouvrir l'examen une seule fois et lire la note avec des mesures médicales.

Explication simple

Le modèle prédit sur les 120 images de test. Convention de ce TP : positif = pathologique. **TP = 31** pathologiques détectés ; **FN = 1** pathologique manqué (le cas grave) ; **FP = 2** sains pris pour pathologiques (fausses alertes) ; **TN = 86** sains bien écartés.

Le code, ligne par ligne

Ligne de code	Ce qu'elle fait, en mots simples
`best = grid.best_estimator_`	Le meilleur modèle trouvé en partie 4.
`pred = best.predict(X_te)`	Prédiction 0/1 pour chaque image du test.
`proba = best.predict_proba(X_te)[:, 1]`	Probabilité d'être pathologique (colonne 1), pour le seuil et l'AUC.
`confusion_matrix(y_te, pred).ravel()`	Les 4 compteurs tn, fp, fn, tp.
`recall_score ... tn/(tn+fp) ... roc_auc_score`	Sensibilité = TP / (TP + FN) ; spécificité = TN / (TN + FP) ; AUC : capacité à classer les malades au-dessus des sains.

Ce qu'on obtient. TN 86 · FP 2 · FN 1 · TP 31 — Sensibilité 0,969 · Spécificité 0,977 · Précision 0,939 · F1 0,954 · AUC 0,998.

À quoi ça sert dans le TP. La seule note à citer : sensibilité 0,969, spécificité 0,977, F1 0,954, AUC 0,998. Elle confirme le comportement vu en validation croisée.

Piège à éviter. Surinterpréter un petit test : il n'y a que 32 cas pathologiques. L'incertitude sur la sensibilité est d'environ ± 6 points (un seul patient pèse 3 points). L'AUC très haute vient aussi de ce que les images sont simulées et bien séparées.

Partie 6 · Seuil pour une sensibilité cible

L'idée en une phrase. fixer le seuil d'alerte pour ne pas rater plus de 5 % des malades.

Explication simple

En dépistage, on veut détecter au moins 95 % des cœurs pathologiques. Il faut donc baisser le seuil de probabilité sous 0,5. Mais on ne choisit pas ce seuil en regardant le test : on utilise des probabilités « honnêtes » obtenues sur le **train** par validation croisée, puis on prend le plus grand seuil qui garantit un rappel d'au moins 0,95.

Le code, ligne par ligne

Ligne de code	Ce qu'elle fait, en mots simples
`cross_val_predict(best, X_tr, y_tr, cv=cv, method="predict_proba")`	Pour chaque image du train, une probabilité prédite par un modèle qui ne l'a pas vue pendant l'apprentissage.
`precision_recall_curve(y_tr, p_cv)`	Pour chaque seuil possible : la précision et le rappel obtenus.
`seuil = max(s for s, r in zip(seuils, rap[:-1]) if r >= 0.95)`	Garde les seuils qui atteignent 95 % de rappel et prend le plus grand, donc le moins de fausses alertes possible.
`pm = (proba >= seuil).astype(int)`	Applique ce seuil au test : 1 si la probabilité l'atteint.
`confusion_matrix(y_te, pm).ravel()`	Relit la matrice avec le nouveau seuil.

Ce qu'on obtient. Seuil 0,21 · test : sensibilité 0,969, spécificité 0,966, précision 0,912 · FN = 1, FP = 3.

À quoi ça sert dans le TP. Transformer un objectif médical (« 95 % de détection ») en réglage technique, sans regarder le test. Le seuil trouvé vaut 0,21.

Piège à éviter. Choisir le seuil en regardant le test : la note devient trop belle. Autre surprise ici : sur ce test, le seuil 0,21 ne fait pas mieux que 0,5 (1 malade manqué dans les deux cas) et ajoute 1 fausse alerte. Avec 32 malades seulement, un patient suffit à brouiller la comparaison.

Partie 7 · Où regarde le modèle ?

L'idée en une phrase. masquer des zones de l'image pour voir lesquelles font changer la décision.

Explication simple

C'est la méthode d'**occlusion** : on cache un petit carré de l'image, on regarde de combien la probabilité de « pathologique » baisse. Si elle baisse beaucoup, la zone cachée était importante. En déplaçant le carré partout, on obtient une carte de chaleur. On vérifie ensuite que le modèle s'appuie sur des zones médicalement sensées (la paroi et la cavité), pas sur un détail sans rapport.

Le code, ligne par ligne

Ligne de code	Ce qu'elle fait, en mots simples
`cand = np.where((y_te == 1) & (proba > 0.5))[0]`	Liste les images pathologiques que le modèle reconnaît (proba > 0,5).
`idx = int(cand[np.argmin(proba[cand])])`	Prend la moins évidente (proba la plus basse) : sur une image déjà à 100 %, masquer un carré ne change presque rien.
`im2[i*4:i*4+12, j*4:j*4+12] = img.mean()`	Remplace un carré de 12 × 12 pixels par la couleur moyenne de l'image, en avançant par pas de 4.
`carte[i, j] = base - best.predict_proba(im2[None])[0, 1]`	Mesure la baisse de probabilité : plus elle est grande, plus la zone comptait.
`plt.imshow(carte, cmap="hot")`	Affiche la carte : clair = zone importante.

Ce qu'on obtient. Image n°100 : probabilité de 0,62 ; la baisse maximale en masquant une zone atteint 0,33.

À quoi ça sert dans le TP. Expliquer une décision, vérifier que le modèle regarde au bon endroit et détecter d'éventuels raccourcis (par exemple un texte incrusté dans un coin de l'image). Ici, les zones importantes se trouvent sur le contour de la paroi.

Piège à éviter. Voir la carte comme une preuve de causalité. C'est une indication, pas une explication médicale : elle dépend de la taille du carré, du remplissage choisi et du modèle.

Récapitulatif

Partie	Question	Outil clé	Résultat
1 Exploration	À quoi ressemblent mes images ?	imshow, mean, value counts	600 images, 27 % pathologiques
2 Découpage	Comment garder un examen honnête ?	train_test_split(stratify)	480 train / 120 test
3 Comparaison	Quelles caractéristiques et quel modèle ?	HOG, Pipeline, cross_val_score	SVM F1 = 0,908
4 Optimisation	Quel réglage ?	GridSearchCV	C = 1, gamma = 0,005
5 Évaluation	Quelle performance réelle ?	confusion_matrix, AUC	Sens. 0,969 · Spéc. 0,977
6 Seuil	Comment viser 95 % de détection ?	cross_val_predict, PR curve	Seuil = 0,21
7 Occlusion	Où regarde le modèle ?	masquage + carte de chaleur	Contour de la paroi

Mini-glossaire

Pixel : un point de l'image, avec une valeur de gris.

HOG : description d'une image par la direction de ses contours.

Speckle : bruit granuleux typique des échographies.

Sensibilité : part des malades détectés.

Spécificité : part des sains correctement écartés.

AUC : probabilité qu'un malade reçoive un score plus haut qu'un sain (1 = parfait, 0,5 = hasard).

Occlusion : masquer une zone de l'image pour mesurer son importance.

Fuite de données : quand de l'information du test s'infiltre dans l'entraînement et gonfle les notes.